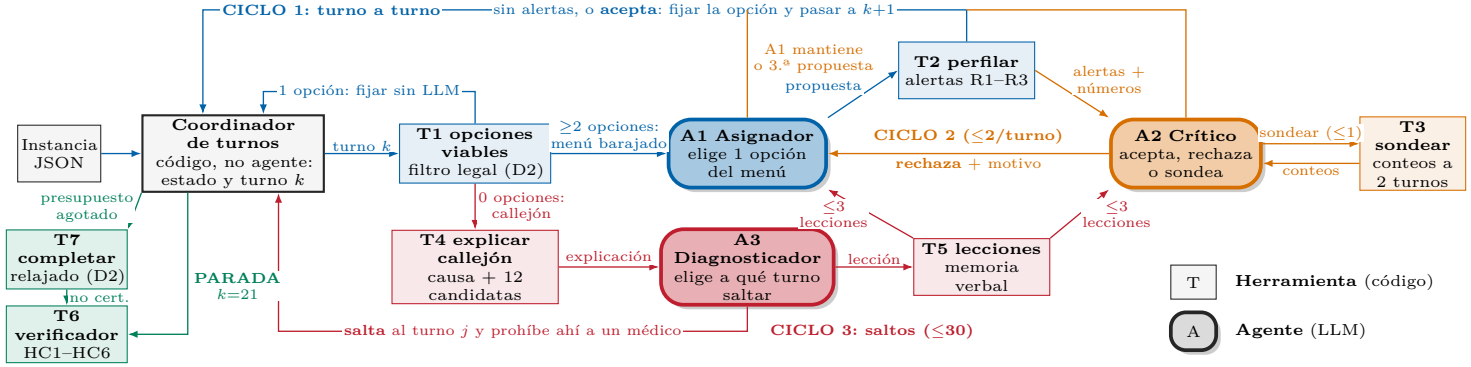
Asignación de Turnos Médicos con un Sistema de Agentes

Deliverable 3 | Generative Artificial Intelligence (580694)

Universidad de Concepción | Grupo 11
Repo: github.com/matirocha/genai-grupo11
Video: VIDEO_LINK



1. Continuidad con D1 y D2, y desviaciones declaradas

Qué se mantiene de D1 y D2:

- la tarea: horario de 10 médicos en 21 turnos (7 días × 3), salida JSON;
- el juez: `verifier.py`; correcto = no viola ninguna regla dura (HC1–HC6);
- el modelo: **Llama-3.2-3B-Instruct** (4-bit, greedy).
- las entradas: 30 principales y 30 de estrés (de D2), más 30 difíciles.

La falla que se ataca: la elección voraz de D1 (F1): gastar temprano las horas y el descanso de los médicos deja el fin de semana imposible. En D2 siguió *dentro* de la elección: una opción legal podía condenar la semana 2–12 turnos antes del callejón, y retroceder de a un turno no llega a esa *decisión culpable*. D3 vuelve directo a ella.

Cómo se construye sobre D2: D3 reutiliza el código de D2 (`stepwise_solver.py`, sin cambios) para filtrar las opciones legales y para completar el horario cuando se acaba el presupuesto. Prueba: si se le quita lo nuevo y elige al azar como D2, entrega los mismos 90 horarios que D2 (`test_continuity.py`).

Desviaciones respecto de D2:

- **Conjunto difícil:** D2 ya resolvía 30/30 del principal: se agregaron 30 instancias con capacidad justa, elegidas por su estructura, no por resultados.
- **Respuestas de 1 token:** el modelo responde siempre un número, el de mayor probabilidad; equivale a la decodificación restringida de D2.
- **Presupuesto:** 30 saltos, como los 30 retrocesos de D2; como un salto deshace hasta 12 decisiones, también se compara con D2 al azar con 300.
- **Ajuste de prompts:** con 20 instancias de calibración aparte, que no están entre las 90 evaluadas.
- **Correcciones:** antes de la corrida final, una revisión del código arregló errores del sistema y de la medición.

2. Las herramientas T1–T7 (`src/tools.py`)

Código sin LLM: calculan números sobre el horario parcial para los agentes.

T1 opciones viables	Opciones para k que cumplen HC1–HC6 y el chequeo hacia adelante (lógica de D2), sin las prohibidas por saltos.
T2 perfilar	Alertas vs. la mejor alternativa: R1 más especialistas sobre la cuota, R2 deja la mitad de opciones o menos, R3 más turnos sin margen.
T3 sondear	Continuaciones viables a 2 turnos (propuesta y alternativa).
T4 explicar callejón	Requisito que ya no se cubre, médicos que faltan y por qué, y las últimas 12 decisiones como candidatas a rehacer.
T5 lecciones	Memoria verbal: dónde fue el callejón, a qué turno se volvió y a quién se prohibió. A1 y A2 leen las 3 últimas.
T6 verificador	`verifier.py`, el juez de D1 y D2: violaciones de HC1–HC6 del horario final (correcto = 0).
T7 completar relajado	Sin presupuesto, completa relajando horas, descanso y luego bloqueos; elige A1 (lógica de D2).

HC1 descanso ≥16 h, **HC2** 1 turno/día, **HC3** dotación, **HC4** cuota de especialidad, **HC5** horas, **HC6** bloqueos.

3. Los tres agentes (prompts versionados en `prompts/`)

Mismo modelo, prompts distintos; cada uno responde un solo número.

Agente	Recibe	Responde	Luego
A1 Asignador	estado, menú barajado (T1), lecciones (T5) y motivo del rechazo	n.º de opción; 0 = mantener	T2 la perfila; sin alertas, se fija
A2 Crítico	propuesta frente a la mejor alternativa, con números de T2 y T3	1 acepta 2 rechaza 3 sondea	acepta: se fija; rechaza: vuelve a A1 (máx. 2)
A3 Diagnosticador	de T4: causa, médicos en conflicto y últimas 12 decisiones	n.º de la decisión a rehacer (j)	deshace $j..k$, prohíbe un médico en j, lección a T5 (máx. 30)

Herramientas: A2 puede llamar a T3 (sondear); A1 y A3 no llaman herramientas: reciben lo que el coordinador obtiene de T1, T4 y T5.

Por qué se divide así: A1 mira a lo ancho; A2 revisa a fondo una propuesta (a un 3B le cuesta menos verificar que generar); A3 mira hacia atrás.

4. Cómo leer el grafo de agentes

Rectángulos: herramientas (código); cajas redondeadas: agentes (LLM). Cada color es un ciclo. Todo lo dirige el **coordinador de turnos** (gris), código que guarda el horario a medio armar y el turno k, y decide a quién le toca. Se llena de a un turno, del lunes AM (k=1) al domingo PM (k=21):

- **Ciclo 1, turno a turno:** T1 da las opciones viables de k. Con 1 se fija sin LLM; con ≥2, A1 elige y T2 la perfila. Sin alertas, se fija y se pasa a k+1.
- **Ciclo 2, revisión:** con alertas decide A2 (puede sondear con T3). Acepta: se fija. Rechaza: vuelve a A1 con el motivo (máx. 2; la 3.ª se fija).
- **Ciclo 3, callejón:** con 0 opciones, T4 lo explica y A3 elige a qué turno j volver: se deshace $j..k$, se prohíbe un médico en j y T5 anota la lección.
- **Parada:** 21 turnos fijados = certificado. Con los 30 saltos gastados, T7 completa relajado (no certificado). T6 juzga ambos.

5. Técnicas de prompting

5a. Qué técnicas usa cada agente (categorías de *The Prompt Report*)

Técnica (*categoría*)	**A1**	**A2**	**A3**	**Falla que ataca**
Role prompting *zero-shot*	✓	✓	✓	F3 de D1: en salidas largas el modelo inventaba IDs de médicos y rompía el JSON.
Salida 1 token *answer engineering*	✓	✓	✓	F3: responde solo un número, así que no puede inventar IDs ni romper el JSON.
Few-shot *few-shot (ICL)*	✓	✓	✓	Elegir mal la opción o el salto; los ejemplos muestran casos ya resueltos.
CRITIC *agentes: tool use*		✓		El chequeo de D2 deja pasar opciones legales que ya condenan la semana.
Self-Refine *self-criticism*	✓			La misma: A1 rehace su propuesta con el motivo del rechazo de A2.
ReAct *agentes: observación*			✓	F1: la decisión culpable está 2–12 turnos atrás y retroceder 1, como D2, no llega.
Reflexion *agentes: observación*	✓	✓	✓	Repetir, después de un salto, el mismo gasto que causó el callejón.
Ensamble *ensembling*	✓			Sesgo de posición: D2 elegía los primeros candidatos el 33 % de las veces (azar: 17 %).

5b. Qué pasa al quitar cada técnica (mismas 90 instancias)

Sistema	**Correctos**	**Sin callejón**	**Llamadas al LLM**	**Qué aporta y por qué**
Línea base	0	–	1	
Solución D2	80	42	26	
D3 completo	**90**	**78**	**77**	
− ReAct (sin A3)	82 (−8)	78 (=)	66	**Clave:** sin A3 se retrocede 1 turno, como D2: vuelve F1.
− Few-shot	87 (−3)	67 (−11)	112	**Ayuda:** sin ejemplos, +11 callejones y +45 % llamadas.
− CRITIC+ Self-Refine	89 (−1)	57 (−21)	63	**Ayuda:** A2 frena malas elecciones; sin él, +21 callejones.
− Reflexion	90 (=)	81 (+3)	86	**Mejora saltos:** sin lecciones, sirve el 70 % de los saltos (no el 100 %).
− Ensamble	90 (=)	76 (−2)	48	**No aporta:** casi igual, pero cuesta 60 % más llamadas.

Rol y salida de 1 token son la base: sin ellas, es la línea base (0/90).

6. Resultados (mismas 90 instancias, mismo juez)

Horarios correctos según `verifier.py`. Abajo, *controles*: el mismo sistema con el LLM reemplazado por azar (promedio de 5 corridas) o por reglas fijas.

Sistema	**Princ.**	**Estrés**	**Difícil**	**Total**
Línea base: prompt directo (D1)	0	0	0	0/90
Solución D2	30	28	22	80/90
Sistema de agentes (D3)	**30**	**30**	**30**	**90/90**
Controles: el LLM se reemplaza por azar o por reglas				
C1. D2 eligiendo al azar	29.8	27.4	24.6	81.8
C2. D2 al azar con 300 retrocesos	29.8	28.2	27.0	85.0
C3. D3 con agentes al azar	30	30	30	90.0
C4. D3 con reglas fijas	29	25	20	74

- D3 resuelve los 90 horarios; D2, 80; la línea base, 0. D3 gana en estrés y difícil (30 y 30, contra 28 y 22).
- D2 sin LLM da casi lo mismo (C1: 81.8), y ni con 10 veces más retrocesos alcanza a D3 (C2: 85).
- D3 con agentes al azar también llega a 90 (C3): el total lo da la estructura de D3; el LLM aporta menos callejones (5b).
- Con reglas fijas (C4) baja a 74: saltan a la decisión equivocada.

7. Límites y reproducción

Límites (lo que este trabajo no demuestra): el 90/90 no se debe al LLM: con agentes al azar (C3) también se llega. El LLM aporta menos callejones y mejores saltos (A3 arregló los 27 callejones; al azar, 69 %; `oracle_eval.py`). Con 90 instancias, diferencias de 1 a 3 horarios pueden ser azar.

Reproducción: los pasos del video están en `video/`; las corridas, trazas y tablas, en `results/`; y el paso a paso, en el README.